Sistema gestión clínica

JavaFX MySQL arquitectura en capas

jareth sosa

CONTENT

01 Funcionalidades del sistema

02 Arquitectura y tecnologías

03 Paquetes y capas

01

Funcionalidades del sistema

Autenticación roles

Inicio de sesión

Login por usuario y contraseña con verificación en base de datos y mensaje de error.

Roles del sistema

Dos roles: ADMIN y DOCTOR, acceso diferenciado según permisos.

Ruta post-login

Tras autenticar usa usuario.esDoctor() y enruta a PanelSelector o VentanaPrincipal.

Vista según rol

DOCTOR ve solo sus citas e historias; ADMIN ve todas y más tabs.

Gestión Pacientes

Registro y búsqueda

Formulario modal y tabla con búsqueda para crear y localizar pacientes.

Edición y eliminación

Editar desde modal y eliminar registro desde la tabla con confirmación.

Validación de datos

Nombre, apellido y DNI obligatorios; errores lanzan IllegalArgumentException.

DNI único en BD

La base de datos impone UNIQUE, evita duplicados al insertar.

Gestión Médicos

Funciones principales.

Registro, búsqueda, edición y eliminación de médicos con formularios modales.

Modelo Médico.

Campos: id, nombre, apellido, especialidad, matrícula, teléfono, email.

Validación y servicio.

MedicoService valida datos y delega persistencia; lanza IllegalArgumentException.

Fusnt : Fland

Startcally Name

Speciially

licares: 15009

Phone:

DAO e implementación.

IMedicoDAO declara contrato; MedicoDAO implementa SQL con JDBC y PreparedStatement.

Citas y estados

Estados de cita

PENDIENTE, ATENDIDA y CANCELADA como enum para cada cita.

Vista y gestión

CitaView muestra tabla, búsqueda y formulario modal para CRUD y cambios de estado.

Filtrado por médico

Doctores ven solo sus citas filtradas por medicoId; ADMIN ve todas.

Persistencia y consultas

CitaDAO usa JOINs para traer nombres y JDBC con PreparedStatement.

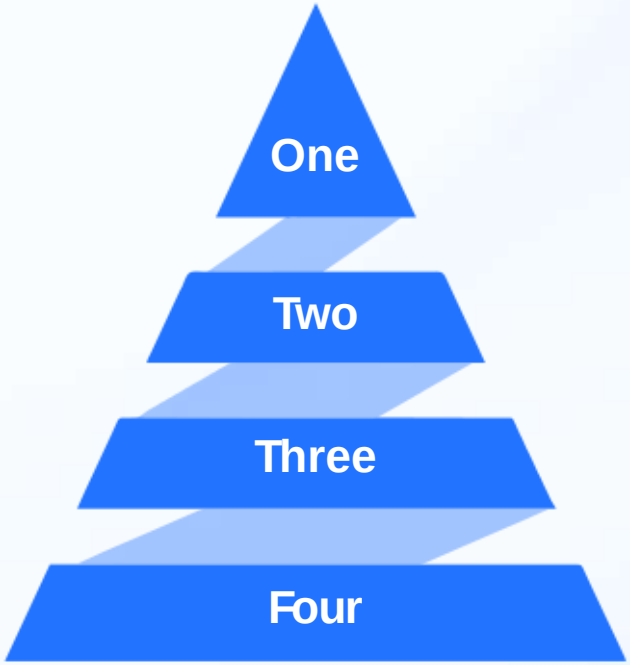
Historias clínicas

Propósito.

Registra diagnósticos, tratamientos y observaciones asociadas a paciente y médico.

Campos clave.

id, pacienteId, medicoId, fecha, diagnóstico, tratamiento y observaciones.



Modelo de datos.

POJO sin lógica, solo datos: HistoriaClinica con campos básicos.

Persistencia (DAO).

DAO interfaz + implementación con SQL/JDBC; lecturas usan JOIN para nombres.

Medicamentos stock

Control de inventario

Registro y seguimiento del stock entero de cada medicamento.

Alerta de stock bajo

Sistema alerta cuando stock alcanza umbral configurable en DashboardDAO.

CRUD medicamentos

Tabla, búsqueda y formulario modal para crear, editar y eliminar.

Modelo Medicamento

Campos: id, nombre, principioActivo, descripcion, stock.

Dashboard y reportes

1

Indicadores clave.

Tarjetas KPI: citas hoy/semana/mes/año, pendientes, pacientes, médicos.

2

Gráfico 14 días.

Barras con mapa citas-por-día para tendencia reciente.

3

Consultas agregadas.

DashboardDAO realiza agregaciones con funciones de fecha MySQL.

4

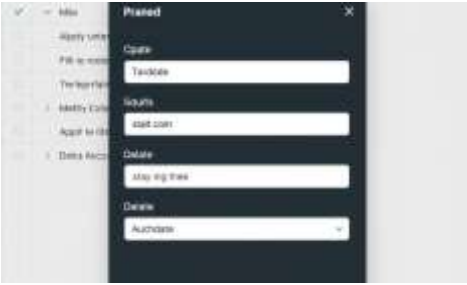
Informes filtrables.

Reportes con filtro por rango de fechas y tabla exportable.

Administración usuarios

Funcionalidad principal.

Tabla de usuarios y formulario modal para crear, editar y eliminar cuentas.



Vínculo a médicos.

Asociar cuenta DOCTOR a un médico mediante campo en el formulario modal.

Activar / desactivar.

Toggle para marcar cuentas activas o inactivas inmediatamente.

Validación y errores.

Servicios validan datos; vistas muestran mensajes claros ante errores.

02

Arquitectura y tecnologías

Arquitectura monolítica

Capas del sistema.

Vista, Servicio, Interfaz DAO, DAO concreto y base de datos MySQL.

1

Responsabilidad de capas.

Vista UI; Servicio valida y orquesta; DAO interfaz abstrae; DAO ejecuta SQL.

2

Dirección de dependencias.

Flujo siempre hacia abajo: vistas→servicios→interfaces DAO, nunca a implementaciones.

3

Inversión de dependencias.

Servicios dependen de abstracciones (interfaces DAO) para desacoplar implementaciones.

4

Capas SOLID

01

Responsabilidad Única

Cada clase tiene un solo propósito: DAO acceden BD, Services validan, Vistas muestran UI.

02

Abierto/Cerrado

Agregar base nueva creando DAOs concretos sin cambiar Services o Vistas.

03

Sustitución de Liskov

Implementaciones de DAO intercambiables sin afectar servicios ni vistas.

04

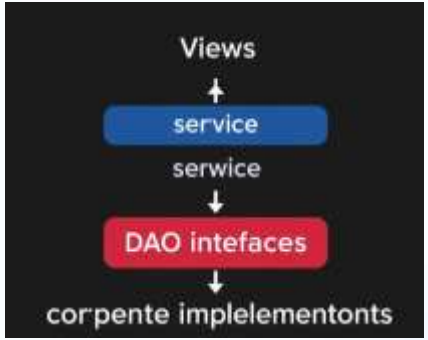
Segregación de Interfaces

Cada interfaz DAO expone solo métodos de su dominio específico.

Inversión dependencias

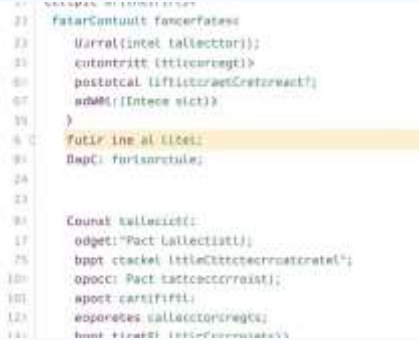
Principio Inversión Dependencias

Servicios dependen de interfaces DAO, no de implementaciones concretas.



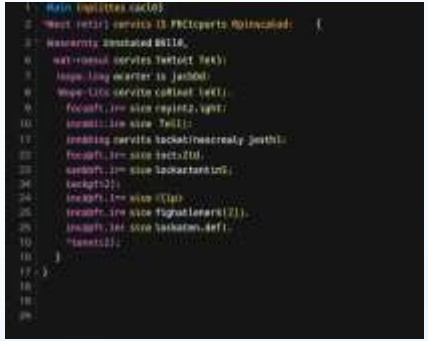
Composition Root centralizado

Main.java instancia las implementaciones concretas y arranca la aplicación.



Flujo de dependencias

Las vistas → servicios → interfaces DAO → implementaciones concretas.



Inyección por constructor

Services reciben interfaces DAO en constructores para mayor flexibilidad.

Composition Root

1

Punto de entrada Main.java

Main.java instancia objetos concretos y controla el inicio de la aplicación.

2

Verificación de conexión

Al iniciar valida la conexión MySQL; muestra error y cierra si falla.

3

Instanciación de DAOs

Se crean siete DAOs concretos: Paciente, Medico, Cita, Usuario, Medicamento, Historia, Dashboard.

4

Inyección de servicios

Cada servicio se instancia inyectando su interfaz DAO correspondiente por constructor.

Java 17

Lenguaje principal para la aplicación de escritorio y la lógica de negocio.

Rol en el proyecto.

JavaFX usado para todas las vistas, extendiendo BorderPane.

Interfaz gráfica JavaFX.

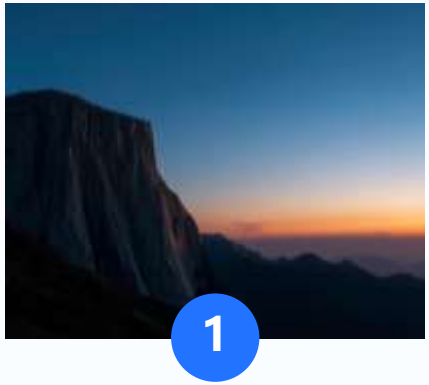
JDBC directo sin ORM, PreparedStatement y try-with-resources.

Acceso a base de datos.

Aplicación monolítica en capas siguiendo SOLID e inversión de dependencias.

Arquitectura y dependencias.

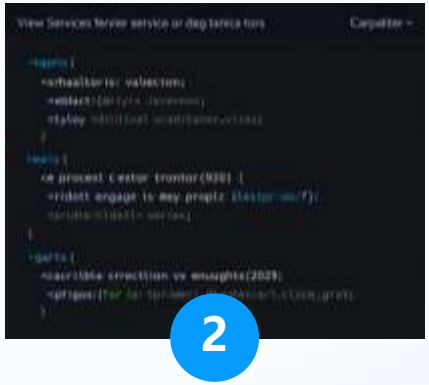
JavaFX UI



1

Rol de JavaFX

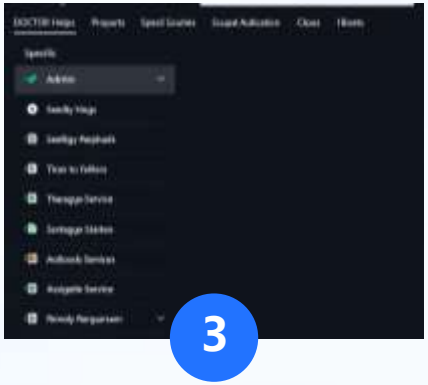
Interfaz de escritorio basada en BorderPane para todas las vistas.



2

Inyección de dependencias

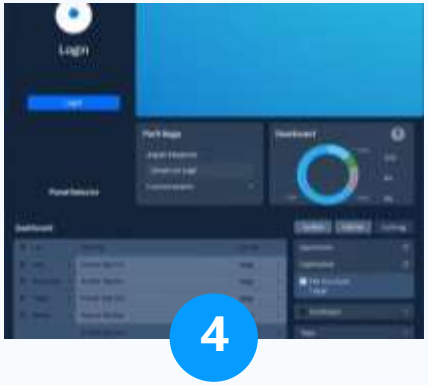
Vistas reciben servicios por constructor, sin crear DAOs ni servicios.



3

Estructura de pantallas

Tabs y navegación según rol: DOCTOR o ADMIN con vistas específicas.



4

Componentes clave

Login, PanelSelector, VentanaPrincipal, Dashboard y vistas CRUD por entidad.

MySQL

01

Rol en la arquitectura

MySQL almacena datos persistentes; capa DAO usa SQL vía JDBC sin ORM.

02

Estructura de la base

Seis tablas: usuarios, pacientes, medicos, citas, historias_clinicas, medicamentos.

03

Restricciones clave

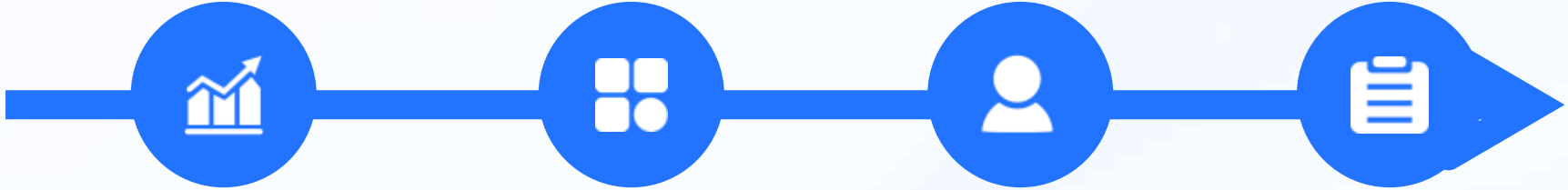
DNI y matrícula con UNIQUE; citas usan ENUM para estado.

04

Conexión y ciclo

Conexion.java es singleton; se crea al iniciar y cierra en Main.stop().

JDBC sin ORM



Acceso BD con JDBC

JDBC usado como único método de persistencia, sin ORM intermedio.

DAO concreto

Clases DAO implementan SQL manualmente usando JDBC.

Prevención SQLi

PreparedStatement con parámetros (?) para evitar inyección SQL.

Auto cierre recursos

Se usa try-with-resources para cerrar conexiones y sentencias automáticamente.

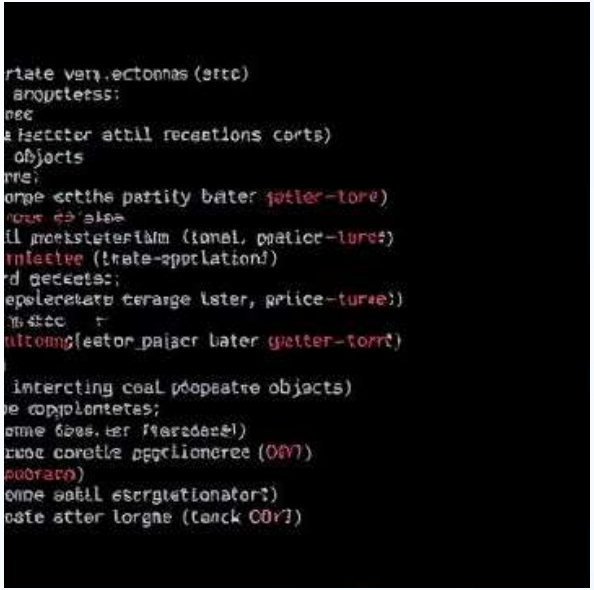
03

Paquetes y capas

Main y AppContexto

Composition Root único

Main.java instancia todos los objetos concretos y orquesta el arranque.



Secuencia de inicio

Verifica conexión MySQL, crea DAOs, services, crea AppContexto, lanza LoginView.

Instanciación de DAOs

Main instancia siete DAOs concretos y los pasa a los servicios.

AppContexto como contenedor

Contiene siete servicios públicos finales para transportar dependencias entre vistas.

Conexion singleton

Patrón Singleton

Conexion.java mantiene una única instancia compartida de conexión MySQL.

Creación al iniciar

Se crea una vez en Main.start(), verificando conexión antes de continuar.

Acceso desde DAOs

Los DAOs llaman Conexion.obtener() para obtener la conexión JDBC compartida.

Cierre en apagado

La conexión se cierra en Main.stop() al cerrar la aplicación.

Modelo POJOs

Usuario.

id, username, password, nombreCompleto, rol, activo, medicoId; esDoctor() identifica doctores.

1

Paciente.

id, nombre, apellido, dni, fechaNacimiento, telefono, email, direccion; DNI único.

2

Medico.

id, nombre, apellido, especialidad, matricula, telefono, email; matrícula única.

3

Cita.

id, pacienteId, medicoId, fechaHora, motivo, estado; PENDIENTE/ATENDIDA/CANCELADA.

4

Interfaces DAO

Contrato de persistencia.

Definen métodos (listar, buscar, insertar, actualizar, eliminar) que forman el contrato de persistencia.

Implementación concreta.

Clases DAO usan SQL y JDBC para implementar las operaciones definidas en las interfaces.

Prácticas seguras SQL.

Usan PreparedStatement y parámetros para prevenir inyección SQL.

Gestión de recursos.

try-with-resources cierra automáticamente conexiones y statements.

Implementaciones DAO

Contrato e implementación

Cada entidad define interfaz DAO; clase concreta implementa SQL con JDBC.

Seguridad SQL

Uso de PreparedStatement para prevenir inyección y parámetros seguros.

```
pUoet: (carl)
  <distrract Peatt(ive, - ():
   votteripg Castts Perlivit, - ));
    calk witk ecytool);
    <clust idl fertivel,, tock(;
      copk zs Ocbet terlitrt, Ferlicl):
      jormAst Desfitture, - ([;
   ];

cUoet:
 <came-is treet (ealtchl();
   <odecsipg(Peatts fenlil @alichl);
    <tapyAst is tesytchl):
      vettly: Lande(Fechetl Otture, - [];
      jercitth):
   ];
```

Gestión de recursos

try-with-resources cierra automáticamente conexiones, statements y resultsets.

Acceso a conexión

Conexion singleton provee la conexión MySQL compartida en toda la app.

Servicios validación

Rol del servicio

Valida datos y orquesta operaciones delegando persistencia al DAO.

Inyección de DAOs

Recibe interfaces DAO por constructor, no implementaciones concretas.

Validación de datos

Lanza IllegalArgumentException con mensajes descriptivos si hay errores.

Insertar o actualizar

Comprueba id==0 para decidir entre insertar o actualizar.

Vistas JavaFX

Extensión BorderPane.

Todas las vistas extienden BorderPane para organizar UI en top/center/bottom/left/right.

Navegación entre pantallas.

Login → PanelSelector o VentanaPrincipal según rol autenticado.

Inyección de dependencias.

Vistas reciben servicios por constructor, sin instanciar DAOs ni servicios.

Control de roles.

DOCTOR ve solo sus citas e historias; ADMIN accede a todas las pestañas.

Navegación por roles

PanelSelector para ADMIN

Pantalla permite elegir AdminUsuarios o VentanaPrincipal según necesidad.

Vista para DOCTOR

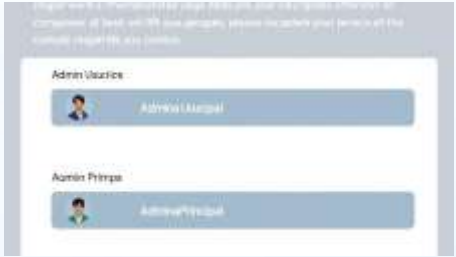
Acceso limitado: solo citas e historias filtradas por medicoId.

01

Autenticación y enrutamiento

Login verifica usuario y redirige según usuario.esDoctor() a DOCTOR o ADMIN.

02



03

VentanaPrincipal y pestañas

Construye tabs distintos según rol, con encabezado y botón cerrar sesión.

04

Base datos tablas

Nombre de la base

La base se llama "clinica" y contiene seis tablas principales.

Tabla usuarios

Cuentas con rol ADMIN/DOCTOR, activo y FK opcional a médicos.

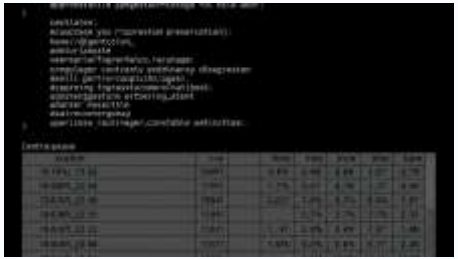
Tabla pacientes

Datos personales; DNI con restricción UNIQUE en la base.

Tabla médicos

Datos del médico; matrícula única con restricción UNIQUE.

DashboardDAO agregación



Conteo de citas

Agrupa y cuenta citas por período (día, semana, mes, año) para KPIs.

Datos para gráfico

Genera mapa citas-por-día de 14 días para el gráfico de tendencia.

01

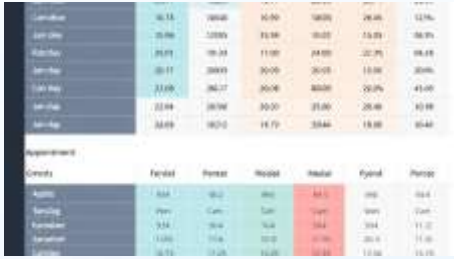
02

03

04

Rol de DashboardDAO

Realiza solo consultas de agregación para estadísticas del sistema, no operaciones CRUD.



Uso funciones MySQL

Emplea CURDATE(), YEARWEEK(), MONTH() para filtrar y agrupar por fecha.

Thanks !